Yes — I'll continue with **Week 2** using the uploaded files:

- **Session 3:** Data Preprocessing, Missing Data & Category Encoding
- **Session 4:** Binary vs Multi-class + Model Complexity
- **Practical:** sklearn preprocessing framework with pipelines, imputation, kNN regression and scaling

Below are your **Week 2 exam notes** in the same two formats: **Structured & Formal Summary** and **Q&A Summary**.

Week 2 — Real-World Model Use and Data Challenges

Part A — Structured & Formal Summary

1. Purpose of Week 2

Week 2 focuses on the messy parts of real-world machine learning that happen before and around model training. The central message is:

A model is only suitable if it can handle the data you give it, or if you correctly transform the data before using the model.

The lecture revisits the comparison framework from Week 1 and focuses on these model capability questions:

- Can the model handle missing values?
- Can it handle continuous, binary, nominal and ordinal inputs?
- Can it handle inputs on arbitrary scales?
- Can it generate probabilities?
- Can it handle binary and multi-class classification?
- How efficient is it in training and prediction?

The lecture stresses that different models accept different kinds of input, so you must know whether to **change the data** through preprocessing or **change the model** to one that handles the data natively.

Session 3 — Data Preprocessing, Missing Data and Category Encoding

2. Why Preprocessing Matters

Preprocessing is the process of transforming raw data into a form that a machine learning algorithm can use correctly.

Common preprocessing problems include:

Problem	Typical Solution
Missing values	deletion, imputation, unknown category, domain-specific filling
Nominal/categorical features	one-hot encoding
Features on different scales	standardisation or normalisation
Model-specific feature requirements	transformation required by the chosen algorithm

The lecture's summary states that these "change the data" solutions require learning a mapping or transformation on the training data and then reapplying the same transformation to future data. Alternatively, you can change the model, but then you must understand the model's implicit strategy.

---

3. Missing Data: Why Most Algorithms Cannot Handle It

Most algorithms cannot directly handle missing values.

The lecture gives two intuitive reasons:

1. **There is no mathematical concept of a NULL number.**
   Algorithms operate on numbers, distances, probabilities, splits or equations. A missing value cannot simply be treated as a normal numerical value.
2. **Many algorithms treat data as points in a geometric space.**
   If one coordinate is unknown, the algorithm may not know where to place the point. For example, if calorie intake is known but exercise hours are unknown, the point cannot be placed cleanly on a two-dimensional obesity-classification graph.

---

4. Why Missing Data Matters in Real-World ML

Missing data affects three major goals of machine learning:

4.1 Generalised Predictive Performance

Generalised predictive performance means how well the algorithm performs on unseen data. Missing data makes it harder to learn and predict because information is missing.

Traditional model building assumes a representative, unbiased sample. Hold-out and cross-validation scores depend on this assumption. If the training data is no longer representative because of missing data handling, real-world performance may differ from validation results.

4.2 Fairness

Missing data can disrupt a representative sample depending on how it is handled.

Example from the slides: older people may be less likely to report their age. If rows with missing age are deleted, older people are removed from the training data. The model may then classify older people poorly.

4.3 Model or Problem Understanding

Missing data can distort conclusions about the problem.

Example from the slides: suppose a model is trained to predict product uptake using age and gender. If men often do not report age and those rows are deleted, the analyst may wrongly conclude that younger people buy the product more. In reality, the pattern may only hold for women.

---

5. Missing Data Mechanisms

The lecture explains that when a value is missing, we do not only lack the value. We also do not know why it is missing. Knowing why values are missing is important because it affects how safe different fixes are.

There are three main types:

---

6. Missing Completely at Random — MCAR

MCAR means there is no relationship between whether a value is missing and any observed or missing values in the dataset.

The lecture describes it as “missing totally at random.”

Example: customer annual income is missing because of random data-entry errors when transferring manual forms to a database. The missingness has nothing to do with the customer being studied.

Implication

MCAR is the safest missingness mechanism. Deleting rows may be acceptable if enough data remains, because deletion should not systematically distort the distribution.

---

7. Missing at Random — MAR

MAR means the data is missing at random within subpopulations defined by observed variables.

The lecture describes it as “missing conditionally at random.”

Example: people aged 30–40 are more likely to leave income blank than other groups, but within that age group the missingness is random. The missingness relates to observed information about the person.

Implication

MAR is more dangerous than MCAR. Deleting rows can remove or underrepresent certain subgroups. Imputation using other variables is often preferable.

---

8. Missing Not at Random — MNAR

MNAR means values are not missing at random. The missingness itself tells us something important that we have not fully observed.

The lecture gives two ways MNAR can occur:

1. Missingness is caused by an unobserved factor.
   Example: people from Australia tend not to report age, but country was not measured. If Australian citizens are older on average, missing age changes the age distribution.
2. People censor their responses.
   Example: very tall or very short people do not report height, or very high and very low earners do not report income.

Implication

MNAR is the hardest case. Deletion and simple imputation can create serious bias. You may need domain knowledge, a missing-value indicator, extra data collection, or careful model-specific handling.

---

9. Key Rule for Missing Data

The lecture gives an important practical rule:

Make a good guess, check performance, check fairness, and check stability of model/problem understanding.

Often you do not know whether data is MCAR, MAR or MNAR. Therefore, you should examine missingness patterns visually or analytically and justify your choice.

---

10. Ways to Handle Missing Data

Missing data can be handled in two broad ways:

1. **Fixed before model input through preprocessing**
2. **Accepted natively by the algorithm**

The lecture stresses that even if an algorithm accepts missing values natively, you must understand how it handles them and whether that strategy is appropriate for the problem.

---

11. Preprocessing Option 1: Deletion

There are two deletion strategies:

11.1 Delete rows with missing values

This removes all data points that contain missing values.

Advantages:

- simple,
- sometimes safe under MCAR,
- avoids guessing missing values.

Risks:

- less training data,
- higher chance of overfitting if data becomes small,
- weaker generalisation,
- possible bias if deletion removes a subgroup.

The lecture gives examples such as unintentionally losing Australian customers, very tall/short people, or low/high income earners.

11.2 Delete features with missing values

This removes entire input variables that contain missing values.

Advantages:

- simple,
- avoids unreliable imputation.

Risks:

- loss of information,
- reduced predictive accuracy,
- may remove an important feature.

The lecture summarises this as “less information” and asks whether predictive accuracy will suffer — probably yes.

---

12. Preprocessing Option 2: Imputation

Imputation means replacing missing values with estimated values.

12.1 Simple imputation

Examples:

- mean,
- median,
- mode.

This is easy but may be misleading, especially under MNAR.

12.2 Model-based imputation

Examples:

- linear regression,
- kNN,
- maximum likelihood.

This predicts missing values from non-missing values. It can use partial information more effectively than simple statistics, but it can still fail if the true missingness mechanism is MNAR.

12.3 Multiple imputation

Multiple imputation creates several datasets with plausible missing values, trains/tests models on each, and examines how results vary. It is useful when missing data may strongly affect accuracy or interpretation, especially with limited data. The lecture says this is slightly outside the course scope, but you should know it exists.

---

13. Preprocessing Option 3: Other Methods

Other missing-data fixes include:

13.1 Create an “Unknown” category

For categorical data, missing can be coded as a separate category.

Example:

- Male
- Female
- Unknown

This is useful when missingness itself may carry information, but it also creates the usual issues associated with categorical variables.

13.2 Use domain knowledge

Example: if there are no visits to a store in January, total spend can be set to 0.

This is not generic imputation. It is a domain-specific correction where the missing value has a logical meaning.

---

14. Simplified Guidelines for Missing Data

The lecture provides oversimplified but exam-useful guidelines:

MCAR

- If lots of data and a feature is missing for many rows: drop the feature.
- If lots of data and feature is missing for few rows: drop data points.
- If little data: imputation is often better.
- Deletion is often reasonably safe because missingness is random.

MAR

- Deletion may cause systematic issues.
- Imputation is often preferable.
- Model/ML-based imputation is usually better than simple imputation.
- Be careful if deletion removes a subpopulation.

MNAR

- Be very careful case by case.
- Coding as missing, using domain knowledge, or kNN-style imputation may be useful.
- Dropping features may be needed if many values are missing.
- Hidden causes should be considered: could the missingness be caused by a factor not measured?

---

15. Algorithms That Handle Missing Values Natively

Some models can accept missing values directly, but this does not mean missingness is magically solved.

The lecture says algorithms can handle missing values when the designers believed there was a logical default. However, that default may or may not be suitable for your application.

---

16. Decision Trees and Missing Values

Decision trees may handle missing values in several implementation-dependent ways:

During training

Possible strategies include:

1. **Surrogate features**
   Use another feature that makes a similar split.
2. **Avoid variables with missing values**
   Only split on variables that do not contain missing values.
3. **Use statistics from complete cases**
   Assign missing-value rows to child nodes using rules such as:
   - send all to majority node,
   - distribute among children with reduced weights,

- randomly assign to child nodes.

During prediction

If a feature value is missing at a split, the tree may:

- use a surrogate split,
- send the instance to the child node with most instances,
- distribute the instance probabilistically or by weighted rules.

The lecture stresses that these strategies are implementation-specific and may affect performance, fairness and model understanding.

Feature Types and Encoding

17. Four Feature Types

The lecture identifies four main input types:

1. **Continuous**
2. **Binary**
3. **Nominal**
4. **Ordinal**

It also notes that “categorical” is used differently by different people. In this module, when the lecturer says categorical, it usually means nominal.

---

18. Continuous Features

Continuous features are numeric values that can vary on a scale.

Examples:

- age,
- income,
- temperature,
- distance,
- density,
- alcohol percentage.

Most machine learning algorithms can handle continuous features directly.

---

19. Binary Features

Binary features have two possible values.

Example:

- red/green,
- yes/no,
- male/female,
- purchased/not purchased.

The lecture's colour example shows that binary features can be directly encoded as 0 and 1. Whether red is coded as 0 or 1 does not fundamentally matter because there are only two categories and one threshold can separate them.

---

20. Nominal Features

Nominal features are unordered categories.

Examples:

- colour: red, green, blue,
- city,
- product category,
- brand,
- occupation.

Nominal features cannot be directly encoded as numbers such as 0, 1, 2 because the numeric values create artificial order and distance.

The lecture demonstrates this with colour: changing the arbitrary encoding of red, green and blue changes what the model may learn for blue, even though the real data has not changed. That is bad because the result depends on an arbitrary coding choice.

---

21. One-Hot Encoding

Nominal features must usually be converted into several binary features.

Example: Colour with red, green and blue becomes:

Red	Green	Blue
1	0	0
0	1	0
0	0	1

This is called **one-hot encoding**.

The lecture says nominal features must be re-encoded as many binary features so that the order of the axis no longer matters.

---

22. One-Hot Encoding and Reference Classes

One-hot encoding can create redundant columns.

Example: if someone must be male or female, then knowing “not female” already means male. Both columns together contain complete redundancy.

This problem is called **multicollinearity**.

To reduce redundancy, one category can be removed. This is called using a **reference class**.

For example, instead of Red, Green and Blue columns, you may keep only Blue and Green. If both are 0, the item must be Red.

When reference classes matter

Use a reference class when:

- the algorithm is affected by multicollinearity,
- you care about model interpretation,
- you are using models where coefficients must be interpreted carefully.

If you only care about prediction, the lecture says there may be less need to worry.

---

23. Problems with One-Hot Encoding

One-hot encoding has costs:

- number of input features increases,
- computational complexity increases,
- optimisation may become harder,
- high-cardinality features can become impractical,
- if a feature has too many categories, you should question whether it is useful or whether a different model is needed.

The lecture gives the example that one name variable could become 10 one-hot variables.

---

24. Ordinal Features

Ordinal features have a meaningful order but uncertain spacing.

Example:

- very good,
- good,
- bad,
- very bad.

They can be encoded in several ways:

24.1 As nominal

This loses ordering information.

24.2 As continuous

This preserves order but assumes exact distances between ranks. For example, very good, good, bad, very bad could be coded as 1, 2, 3, 4 or perhaps 1, 2, 4, 5. The correct spacing is not always obvious.

The lecture's takeaway: ordinal features are often encoded as continuous, but you should know that encoding as nominal or using specialised ordinal algorithms is also possible.

---

25. Models That Handle Categorical Values Natively

Some decision-tree implementations can handle categorical variables directly, but this depends on the implementation.

The lecture gives three possible approaches:

1. Not handle categorical variables directly.
2. One-vs-rest splitting.
3. One child per value.

It notes that sklearn decision trees traditionally do not directly handle nominal categorical values in the same way some other implementations do, so preprocessing may be required.

Feature Scaling

26. Why Features on Different Scales Can Be a Problem

Features on different scales can badly affect models that use distance measures.

Example: kNN with L1 or L2 distance.

If one variable ranges from 0 to 100,000 and another ranges from 0 to 1, the large-scale variable may dominate the distance calculation even if it is not more important.

The lecture warns that this may be a problem after one-hot encoding too, because binary features and continuous features can be on very different scales.

---

27. Standardisation

Standardisation rescales features to have:

- mean 0,
- unit variance.

It is useful when a model needs features to be comparable in spread.

Plain formula idea:

standardised value = value minus training mean, divided by training standard deviation.

The lecture states that standardisation rescales data to have mean 0 and unit variance.

---

28. Normalisation

Normalisation rescales data to a fixed range, usually 0 to 1.

The lecture warns that outliers can drastically affect normalisation. It should be used when the specific model or problem requires it.

---

29. Training/Test Consistency in Preprocessing

The same transformation must be used on both training data and test/future data.

Important rule:

Fit the preprocessing step on the training data only, then apply it to test/future data.

Why?

If you fit the transformation on the test data too, you leak information from the test set into the training process. Also, changing the transformation changes the meaning of the model input and can make the trained model invalid.

---

30. Models That Natively Account for Scale

Some models can account for scale through the way distance is defined.

Example from the lecture: kNN with Mahalanobis distance can be equivalent to standardising features and using Euclidean distance.

But the lecture's key question remains:

Is this the right thing for your problem?

---

sklearn Pipelines

31. Why Use Pipelines?

Preprocessing steps such as imputation, one-hot encoding and scaling must be learned on training data and reapplied consistently to future data.

A pipeline integrates preprocessing and modelling into a single workflow.

Benefits:

- avoids manual repetition,

- reduces risk of data leakage,
- ensures preprocessing is fitted only inside each training fold during cross-validation,
- makes the whole workflow behave like one model,
- allows `cross_val_score()` to evaluate preprocessing plus model together.

---

32. sklearn Model Interface

Models generally follow this structure:

- `fit(data, target)` learns model parameters.
- `predict(data)` generates predictions.

33. sklearn Preprocessor Interface

Preprocessors generally follow this structure:

- `fit(data)` learns transformation parameters.
- `transform(data)` applies the transformation.

Examples:

- imputer learns replacement values or neighbour relationships,
- scaler learns means and standard deviations,
- one-hot encoder learns category mapping.

The lecture explains that pipelines chain preprocessors and then feed transformed data into a model.

---

34. Practical Example: Wine Quality Prediction

The practical uses a modified wine quality regression problem. The task is to predict wine quality score from physicochemical measurements such as acidity, residual sugar, chlorides, density, pH, sulphates and alcohol.

The practical identifies missing values:

- `residual sugar` has 14 missing values,
- other variables have no missing values.

The practical uses `missingno.matrix()` to visualise missingness, then sorts by `density` and observes that the missing values are not random. The notes conclude this implies **MAR, not MCAR**, so dropping instances is likely a bad idea because it may remove high-density wines and harm the model's ability to predict that class of wine.

---

35. Practical Pipeline: Imputation and kNN

The practical uses:

- `KNNImputer`,
- `DummyRegressor`,
- `KNeighborsRegressor`,
- `StandardScaler`,
- `Pipeline`,
- `KFold`,
- `cross_val_score()`.

The input and output are separated:

- `y = data.quality`
- `X = data.drop('quality', axis=1)`

The imputer is:

`KNNImputer(n_neighbors=2, weights="uniform")`

Then pipelines are created:

- dummy pipeline: imputation plus dummy regressor,
- kNN pipeline: imputation plus kNN regressor,
- standardised kNN pipeline: imputation, scaling, kNN regressor.

The practical shows the following MAE results:

Model	MAE
Dummy regressor	0.6834
kNN without standardisation	0.4834
kNN with standardisation	0.4158

The key result is that standardising inputs improves kNN because kNN relies on distance calculations, and unscaled features distort distance.

Session 4 — Binary vs Multi-Class and Model Complexity

36. Binary Classification

Binary classification decides whether a point belongs to a class or not.

Example:

- fraud / not fraud,

- churn / not churn,
- authentic / not authentic.

The target has two classes.

---

37. Multi-Class Classification

Multi-class classification decides which one, and only one, of several classes a point belongs to.

Example:

- eye colour: blue, green, brown,
- product category,
- customer segment.

The lecture states that some classifiers are multi-class by design, while others are not.

---

38. Models That Are Multi-Class by Design

The lecture lists these as multi-class by design:

- kNN,
- Decision Tree,
- Naive Bayes.

These can naturally choose among more than two classes.

---

39. Models That Are Natively Binary

The lecture lists these as not naturally multi-class:

- Logistic Regression,
- SVM, linear or kernel.

These can be adapted to multi-class problems using strategies such as one-vs-rest or one-vs-one.

---

40. One-vs-Rest / One-vs-All

One-vs-Rest, also called One-vs-All, creates one binary classifier per class.

For N classes:

- train N classifiers,
- each classifier predicts whether the instance belongs to that class or all other classes,
- final prediction is the class with the highest score or probability.

Example: eye colour with Blue, Green, Brown.

- classifier 1: Blue vs Green/Brown,
- classifier 2: Green vs Blue/Brown,
- classifier 3: Brown vs Blue/Green.

One-vs-Rest usually requires each binary classifier to produce a score or probability.

---

41. One-vs-One / All-vs-All

One-vs-One trains one binary classifier per pair of classes.

For N classes:

number of classifiers = N(N-1)/2

Example: Blue, Green, Brown gives:

- Blue vs Green,
- Blue vs Brown,
- Green vs Brown.

At prediction time, all classifiers vote, and the class with the most votes is selected.

Unlike One-vs-Rest, One-vs-One does not require probabilistic output.

---

42. Choosing OvR or OvO

The lecture says the choice between OvR and OvO is largely computational. In practice, it is often reasonable to use the default strategy provided by the implementation because it may be coded efficiently.

A binary model adapted carefully to multi-class, such as a tuned OvR SVM, can sometimes beat a model that handles multi-class natively, such as a Random Forest, depending on the problem. However, compute cost also matters.

---

43. Models That Output Probabilities

Some models are probabilistic by design.

Example:

- Naive Bayes.

Other models can be modified to produce outputs with probabilistic interpretation.

Examples from the lecture:

1. **SVM distances to probabilities**
   SVM distances from the hyperplane can be converted into probabilities, often using Platt

scaling.

2. **Neural network outputs to probabilities**
   Neural network real-valued outputs can be converted to probabilities through a softmax final layer.

Model Efficiency and Complexity

44. Why Efficiency Matters

When choosing a model, you must ask:

- How long does it take to train?
- How long does each prediction take?
- Is it possible to train and use the model with the expected amount of data?

Efficiency matters in real systems. For example, if a model must recommend an offer at a checkout in real time, prediction speed is crucial.

---

45. Time Efficiency and Space Efficiency

Algorithms can be described in terms of:

- **time efficiency**: how runtime grows,
- **space efficiency**: how memory use grows.

The lecture focuses mainly on time efficiency and scaling behaviour as input size grows.

---

46. Why Wall-Clock Time Is Not Enough

You can time an algorithm by comparing system time before and after it runs. This can be useful in practice, but it is not a reliable general comparison of algorithm efficiency.

Runtime is affected by:

- CPU speed,
- RAM,
- specialised hardware,
- operating system,
- system configuration,
- programming language,
- other programs running,
- memory management,
- algorithm implementation.

Therefore, algorithm analysis usually counts operations and describes growth with input size.

---

47. Cost Functions

A cost function expresses runtime as a function of input size.

Example from the lecture: printing every element in an array.

If the array has n elements, the algorithm must loop through n elements. The lecture expresses the cost as:

$t = 3n + 2$

As n becomes large, constants matter less, so this is described as O(n), or linear time.

---

48. Big-O Notation

Big-O notation describes how runtime grows as input size becomes large.

Common classes from the lecture:

Big-O	Name
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential

The lecture explains that constants are ignored because, for large n, they matter much less than the growth pattern.

---

49. Best, Worst and Average Case

Algorithms may have different runtimes depending on input data or input order.

Examples:

- a sorting algorithm may be faster if data is already sorted,
- a search algorithm may be faster if the item is first.

Big-O can describe:

- best case,
- worst case,
- average case.

Computer scientists often discuss worst case; practitioners often care about average case.

---

50. kNN Complexity

The lecture compares two implementations of kNN.

50.1 List-based kNN

Training:

- O(1)

Prediction:

- O(nd)

where:

- n = number of data points,
- d = number of features.

This means list-based kNN is cheap to train but expensive to predict, because each prediction compares the new point to stored training points.

50.2 Tree-based kNN

Training:

- O(dnlogn)

Prediction:

- O(dlogn)

Tree-based kNN has more training cost but faster prediction.

---

51. Random Forest Complexity

The lecture gives Random Forest complexity as:

Training:

- $O(MK \log^2 n)$

Prediction:

- best case: O(Mlogn)
- worst case: O(Mn)

where:

- n = number of data points,

- d = number of features,
- M = number of trees,
- K = number of features tried at each node.

The lecture notes that both Random Forest and kNN are relatively efficient compared with some later algorithms such as kernel SVMs.

---

52. Practical Efficiency Choice

A model is not just selected by accuracy. Real-world choice depends on:

- training time,
- prediction time,
- number of data points,
- number of features,
- whether prediction must happen in real time,
- whether training can happen offline,
- memory constraints,
- interpretability,
- expected data growth.

The final Session 4 exercise asks students to choose between list-based and tree-based kNN for real-time checkout offer prediction. Since prediction must happen in real time, tree-based kNN is more appropriate despite higher training cost, because its prediction time is lower.

Week 2 High-Yield Exam Checklist

You should be able to explain:

- why missing data affects performance, fairness and understanding,
- the difference between MCAR, MAR and MNAR,
- when deletion is risky,
- when imputation is preferable,
- why domain knowledge can be better than generic imputation,
- why nominal features cannot be encoded as arbitrary numbers,
- how one-hot encoding works,
- what multicollinearity means in one-hot encoding,
- when to use a reference class,
- why ordinal features are tricky,
- why scaling matters for distance-based models,
- difference between standardisation and normalisation,

- why preprocessing must be fitted on training data only,
- why pipelines prevent leakage and simplify cross-validation,
- how binary classifiers can be adapted to multi-class problems,
- difference between OvR and OvO,
- why output probabilities may matter,
- how Big-O helps compare algorithms,
- why kNN has cheap training but potentially expensive prediction,
- why model choice must consider both accuracy and efficiency.

Part B — Q&A Summary

1. What is the main focus of Week 2?

Week 2 focuses on real-world data problems: missing values, input feature types, categorical encoding, feature scaling, sklearn pipelines, binary versus multi-class classification, probability outputs and model efficiency.

---

2. Why do different models need different preprocessing?

Because models differ in what inputs they can handle. Some models only accept continuous numeric data, some cannot handle missing values, some are sensitive to scale, and some require specific encodings.

---

3. What should you ask before using a model?

Ask whether the model can handle the input data directly. If not, decide whether to transform the data or choose a different model.

---

4. Why can most algorithms not handle missing values?

Because most algorithms operate mathematically on numbers or geometric points. A missing value is not a number, and a point with an unknown coordinate cannot be placed clearly in feature space.

---

5. What three goals can missing data affect?

Missing data can affect predictive performance, fairness, and model/problem understanding.

---

6. How does missing data affect predictive performance?

It removes information and may make the sample unrepresentative. A model may perform well in cross-validation but poorly in the real population if missingness handling creates bias.

7. How can missing data affect fairness?

If missingness is more common in a subgroup, deleting missing rows can remove that subgroup from training. The model may then perform poorly for that subgroup.

---

8. How can missing data affect model understanding?

It can distort conclusions. For example, deleting rows with missing age may make it look like younger people buy a product more, when the true relationship depends on another variable such as gender.

---

9. What does MCAR mean?

MCAR means Missing Completely at Random. Missingness has no relationship with any observed or missing values in the dataset.

---

10. Give an example of MCAR.

A customer's income is missing because of random data-entry errors when transferring paper forms to a database.

---

11. Why is MCAR the safest missingness type?

Because deletion should not systematically change the distribution of the data if missingness is truly random.

---

12. What does MAR mean?

MAR means Missing at Random. Within subgroups defined by observed variables, missingness is random.

---

13. Give an example of MAR.

People aged 30–40 are more likely to leave income blank than other age groups, but within that age group the missingness is random.

---

14. Why can deletion be risky under MAR?

Because it may remove or underrepresent specific observed subgroups, causing biased model performance or interpretation.

---

15. What does MNAR mean?

MNAR means Missing Not at Random. Missingness itself is related to unobserved information or the missing value itself.

---

16. Give an example of MNAR.

Very high and very low earners may choose not to report income. The missingness depends on the value that is missing.

---

17. Why is MNAR difficult?

Because the reason for missingness is not fully captured in the observed data. Simple deletion or imputation may introduce serious bias.

---

18. What is the key practical rule for missing data?

Make a good guess, check model performance, check fairness, and check whether model/problem understanding remains stable.

---

19. What are the main ways to fix missing data in preprocessing?

Deletion, imputation, and other methods such as using an Unknown category or applying domain knowledge.

---

20. What is row deletion?

It removes all data points containing missing values.

---

21. When is row deletion safer?

It is safer when missingness is MCAR and there is enough remaining data.

---

22. Why can row deletion be harmful?

It reduces training data and may remove important subpopulations, causing poor generalisation or unfair predictions.

---

23. What is feature deletion?

It removes an input feature that contains missing values.

---

24. What is the main risk of feature deletion?

It removes information and may reduce predictive accuracy.

---

25. What is imputation?

Imputation replaces missing values with estimated values.

---

26. What are simple imputation methods?

Mean, median and mode imputation.

---

27. What are model-based imputation methods?

Methods that predict missing values using other variables, such as linear regression, kNN or maximum likelihood.

---

28. What is multiple imputation?

It creates multiple plausible completed datasets, trains/tests models on each, and checks how sensitive results are to missing values.

---

29. When might an Unknown category be useful?

When a categorical value is missing and the fact that it is missing may itself contain useful information.

---

30. Why can domain knowledge be better than generic imputation?

Because sometimes the missing value has a logical meaning. For example, no store visit in January may mean total spend should be 0.

---

31. Can some algorithms handle missing values natively?

Yes, but you must understand how they do it and whether the strategy is appropriate for your problem.

---

32. How can decision trees handle missing values?

They may use surrogate splits, avoid splitting on variables with missing values, send missing cases to the majority child, distribute them with weights, or use other implementation-specific rules.

---

33. Why must you check the implementation?

Because different libraries handle missing values differently, and the default strategy may affect performance, fairness and interpretation.

---

34. What are the four feature types in Week 2?

Continuous, binary, nominal and ordinal.

---

35. What is a continuous feature?

A numerical feature measured on a scale, such as age, temperature, density or income.

---

36. What is a binary feature?

A feature with two possible values, such as yes/no or red/green.

---

37. Can binary features be encoded as 0 and 1?

Yes. Binary features can usually be encoded directly as 0 and 1.

---

38. What is a nominal feature?

A feature with unordered categories, such as colour, city or product type.

---

39. Why can nominal features not be encoded as 0, 1, 2 directly?

Because the numbers create artificial order and distance. Different arbitrary encodings can lead to different model behaviour.

---

40. What is one-hot encoding?

One-hot encoding converts a nominal feature into multiple binary features, one for each category.

---

41. Why does one-hot encoding solve the nominal encoding problem?

It removes artificial ordering. Each category gets its own binary indicator.

---

42. What is multicollinearity in one-hot encoding?

It occurs when one encoded column can be perfectly predicted from the others. For example, if a person must be red, green or blue, knowing not red and not green implies blue.

---

43. What is a reference class?

A reference class is a category removed from one-hot encoding to avoid perfect redundancy.

---

44. When should you consider using a reference class?

When the algorithm is affected by multicollinearity or when model interpretation matters.

---

45. What is a drawback of one-hot encoding?

It can create many new features, increasing computation and possibly causing optimisation or interpretation issues.

---

46. What is an ordinal feature?

A feature with ordered categories, such as very bad, bad, good and very good.

---

47. Why are ordinal features tricky?

They have an order, but the exact distance between categories may be unclear.

---

48. How can ordinal features be encoded?

They can be encoded as continuous values, encoded as nominal categories, or handled by specialised ordinal models.

---

49. What is the usual practical approach for ordinal features?

Encode them as continuous, but remember that this assumes distances between ranks.

---

50. Why do features sometimes need scaling?

Because models using distances, such as kNN, can be dominated by variables with larger numerical ranges.

---

51. What is standardisation?

Standardisation rescales a feature to have mean 0 and unit variance.

---

52. What is normalisation?

Normalisation rescales a feature to a fixed range, often 0 to 1.

---

53. Why can normalisation be affected by outliers?

Because extreme minimum or maximum values determine the scaling range.

---

54. Why must preprocessing be fitted only on training data?

Because fitting preprocessing on test data leaks information and changes the meaning of the model input.

---

55. What is a data leakage risk in preprocessing?

Using information from the test set to calculate imputation values, scaling parameters or encoding mappings.

---

56. What is an sklearn pipeline?

A pipeline chains preprocessing steps and a model into one object that can be fitted and used for prediction.

---

57. Why are pipelines useful in cross-validation?

They ensure preprocessing is fitted separately inside each training fold, preventing leakage from validation folds.

---

58. What does a preprocessor's `fit()` do?

It learns transformation parameters, such as means, standard deviations, imputation rules or category mappings.

---

59. What does a preprocessor's `transform()` do?

It applies the learned transformation to data.

---

60. What did the wine practical show?

It showed that using KNNImputer handled missing residual sugar values, and adding StandardScaler improved kNN regression MAE from about 0.4834 to about 0.4158.

---

61. Why did standardisation improve kNN?

Because kNN depends on distances, and standardisation prevents large-scale variables from dominating the distance calculation.

---

62. What is binary classification?

Binary classification predicts one of two possible classes.

---

63. What is multi-class classification?

Multi-class classification predicts exactly one class out of more than two possible classes.

64. Which models are multi-class by design according to the lecture?

kNN, Decision Tree and Naive Bayes.

65. Which models are treated as binary by design in the lecture?

Logistic Regression and SVM.

66. Can binary classifiers be used for multi-class tasks?

Yes. They can be adapted using One-vs-Rest or One-vs-One methods.

67. What is One-vs-Rest?

Train one classifier per class. Each classifier predicts that class versus all other classes. The class with the highest score is selected.

68. How many classifiers does One-vs-Rest train?

N classifiers for N classes.

69. What does One-vs-Rest usually require?

It usually requires each binary classifier to produce a score or probability.

70. What is One-vs-One?

Train one classifier for every pair of classes. All classifiers vote, and the class with the most votes is selected.

71. How many classifiers does One-vs-One train?

N(N-1)/2 classifiers for N classes.

72. Does One-vs-One require probabilities?

No. It can work through voting.

73. Which is better: OvR or OvO?

The lecture says the choice is largely computational, and using the implementation default is often reasonable.

74. What is a probabilistic model?

A model that naturally outputs probabilities for classes.

75. Give an example of a probabilistic model.

Naive Bayes.

76. How can SVM outputs be converted into probabilities?

By converting distances from the hyperplane into probabilities, often using Platt scaling.

77. How can neural network outputs be converted into probabilities?

Using a softmax final layer.

78. What does model efficiency mean?

It refers to how much time and memory a model needs for training and prediction.

79. Why does prediction speed matter?

Because some applications require real-time predictions, such as recommending an offer at checkout.

80. Why is wall-clock time not a perfect comparison?

Because it depends on hardware, operating system, programming language, implementation and other system conditions.

81. What is Big-O notation?

Big-O describes how algorithm runtime or memory use grows as input size becomes large.

82. Why do we ignore constants in Big-O?

Because for large input sizes, growth rate matters more than fixed constants.

83. What does O(1) mean?

Constant time. Runtime does not grow with input size.

84. What does O(n) mean?

Linear time. Runtime grows proportionally with input size.

---

85. What does O(n²) mean?

Quadratic time. Runtime grows with the square of input size.

---

86. What is best-case complexity?

The complexity under the most favourable input conditions.

---

87. What is worst-case complexity?

The complexity under the least favourable input conditions.

---

88. What is average-case complexity?

The expected complexity under typical input conditions.

---

89. What is list-based kNN training complexity?

O(1), because it mostly stores the training data.

---

90. What is list-based kNN prediction complexity?

O(nd), because prediction compares the new point against stored data points across features.

---

91. What is tree-based kNN training complexity?

O(dnlogn), because it builds a tree structure.

---

92. What is tree-based kNN prediction complexity?

O(dlogn), because the tree speeds up neighbour search.

---

93. What is the trade-off between list-based and tree-based kNN?

List-based kNN is faster to train but slower to predict. Tree-based kNN is slower to train but faster to predict.

---

94. Which kNN version is better for real-time checkout predictions?

Tree-based kNN is better because prediction speed matters more than training speed in real-time use.

---

95. What is Random Forest training complexity from the lecture?

$O(MK \log^2 n)$, where M is number of trees, K is number of features tried at each node, and n is number of data points.

---

96. What is Random Forest prediction complexity from the lecture?

Best case $O(M \log n)$, worst case $O(Mn)$.

---

97. What should you consider when choosing a model in real-world use?

Accuracy, preprocessing needs, missing data handling, feature types, scale sensitivity, probability outputs, binary/multi-class support, training time, prediction time, memory and interpretability.

---

98. What is the main exam message of Week 2?

Real-world machine learning is not just model fitting. You must justify how you handle messy data, choose encodings, prevent leakage, use pipelines correctly, and select models based on both predictive performance and practical efficiency.